DATA SCIENCE / GUIA TÉCNICO EP

Pandas

Seleção, limpeza, agregação, junções e arquivos

Objetivo. Estudo aprofundado para apoiar os notebooks de Programação Python, com explicações, exemplos executáveis, exercícios e cuidados para ambientes reais.

Mapa de estudo. 1. Series, DataFrames e seleção • 2. Ausências, tipos e limpeza • 3. Agregação, groupby e junções • 4. CSV, Excel e arquivos locais • 5. Índices e alinhamento automático • 6. Strings, datas e categorizações • 7. Confiabilidade em agregações • 8. Excel, CSV e integridade dos arquivos

Como estudar

Leia o princípio, execute o exemplo em um ambiente preparado, modifique entradas e resolva o desafio antes de avançar. Os exemplos são didáticos e não alteram os notebooks originais.

Relação com o acervo

Este guia é complementar à trilha de seis notebooks. A documentação oficial das bibliotecas é a fonte de referência para APIs e mudanças de versões.

01 Series, DataFrames e seleção

DataFrame combina colunas potencialmente heterogêneas com rótulos; loc seleciona por rótulo e iloc por posição. A avaliação de tipos, dimensões e índices deve preceder operações complexas.

Exemplo executável

```
import pandas as pd
df = pd.DataFrame({"produto": ["A", "B"], "preco": [12, 25]})
assert df.loc[0, "preco"] == 12
assert df.iloc[1, 1] == 25
```

Atenção técnica. Filtros com & e | precisam de parênteses; não use and/or diretamente entre Series booleanas.

Exercício de consolidação. Filtre preços maiores que 15 e selecione apenas duas colunas.

02 Ausências, tipos e limpeza

Valores ausentes exigem interpretação antes da imputação. isna identifica lacunas; fillna e dropna mudam a população analisada. Conversões com pd.to_numeric e pd.to_datetime devem lidar com erros explicitamente.

Exemplo executável

```
df = pd.DataFrame({"valor": [10, None, 30]})
assert df["valor"].isna().sum() == 1
df["valor_limpo"] = df["valor"].fillna(df["valor"].median())
assert df["valor_limpo"].tolist() == [10, 20, 30]
```

Atenção técnica. Preencher dados de teste usando estatísticas calculadas em todo o dataset causa vazamento de informação em modelagem.

Exercício de consolidação. Compare exclusão, imputação pela mediana e indicador de ausência.

03 Agregação, groupby e junções

groupby agrega por chaves; merge une tabelas e pode multiplicar registros quando chaves têm duplicatas. validate declara cardinalidade esperada, indicator ajuda auditar correspondências.

Exemplo executável

```
clientes = pd.DataFrame({"id": [1, 2], "nome": ["Lia", "Caio"]})
pedidos = pd.DataFrame({"id": [1, 1, 2], "valor": [10, 5, 7]})
r = pedidos.merge(clientes, on="id", validate="many_to_one")
assert r.groupby("nome")["valor"].sum()["Lia"] == 15
```

Atenção técnica. Cheque registros sem correspondência e contagem antes/depois do merge; não suponha unicidade das chaves.

Exercício de consolidação. Crie merge com indicator=True e identifique casos left_only.

04 CSV, Excel e arquivos locais

read_csv, read_excel, to_csv e to_excel dependem de encoding, separador, aba e engine. Use caminhos relativos e preserve os originais em outputs antes de executar células que exportam dados.

Exemplo executável

```
from pathlib import Path
p = Path("outputs") / "vendas_simuladas.csv"
if p.exists():
    df = pd.read_csv(p)
    print(df.head())
else:
    print("Execute na pasta da materia")
```

Atenção técnica. No Excel, openpyxl é dependência prática para leitura/escrita de .xlsx; confira nomes de abas e tipos importados.

Exercício de consolidação. Leia um CSV e um XLSX do pacote e compare nomes/tipos de colunas.

05 Índices e alinhamento automático

Series e DataFrames alinham operações usando rótulos. Ao somar Series com índices diferentes, o resultado pode conter NaNs inesperados. Resetar índice muda rótulos, não apenas aparência; use intencionalmente.

Exemplo executável

```
import pandas as pd
a = pd.Series([10,20], index=["x","y"])
b = pd.Series([1,2], index=["y","z"])
r = a + b
assert pd.isna(r["x"]) and r["y"] == 21
```

Atenção técnica. Misturar arrays posicionais com Series rotuladas pode produzir correspondências diferentes da intenção.

Exercício de consolidação. Compare a+b com soma posicional após converter ambos para numpy.

06 Strings, datas e categorizações

Acessores .str e .dt operam por coluna; parse de datas requer atenção a formatos ambíguos e fusos horários. Categorias reduzem memória e expressam domínio em alguns conjuntos, mas exigem política para novos valores.

Exemplo executável

```
df = pd.DataFrame({"data": ["2026-01-15","2026-02-01"]})
df["data"] = pd.to_datetime(df["data"])
df["mes"] = df["data"].dt.month
assert df["mes"].tolist() == [1,2]
```

Atenção técnica. Datas sem fuso não representam automaticamente instantes UTC; trate localizações e conversões explicitamente.

Exercício de consolidação. Extraia ano e mês de uma coluna e agregue contagens mensais.

07 Confiabilidade em agregações

Um resumo deve declarar denominador, tratamento de ausências e nível de agrupamento. groupby pode excluir chaves ausentes conforme configuração. Compare somas por grupo com total original para conferir reconciliação.

Exemplo executável

```
df = pd.DataFrame({"regiao": ["N","S","N"], "venda": [10,20,5]})
r = df.groupby("regiao")["venda"].sum()
assert r.sum() == df["venda"].sum()
assert r["N"] == 15
```

Atenção técnica. Agregações depois de joins muitos-para-muitos podem duplicar fatos e produzir totais incorretos.

Exercício de consolidação. Inclua uma linha sem região e analise dropna=False.

08 Excel, CSV e integridade dos arquivos

Excel pode conter múltiplas planilhas, fórmulas e formatos que não são preservados integralmente por operações tabulares simples. Ao exportar DataFrames, diferencie dados calculados de arquivos originais e verifique encoding e separadores.

Exemplo executável

```
from pathlib import Path
base = Path("outputs")
arquivos = sorted(base.glob("*.xlsx"))
print([x.name for x in arquivos])
# Execute leitura somente apos verificar o arquivo desejado.
```

Atenção técnica. Não regrave planilhas históricas na mesma localização sem backup; fórmulas, formatação e abas podem ser perdidas.

Exercício de consolidação. Selecione duas abas de um XLSX, compare colunas e registre limitações.

Checklist de revisão

Verifique	Evidência
Conceito	Consegue explicar sem copiar o código?
Execução	O exemplo produz o resultado esperado?
Casos-limite	Entradas inválidas e extremas foram testadas?
Reprodutibilidade	O kernel e as dependências estão registrados?

Referências técnicas

Python: docs.python.org/3/ • NumPy: numpy.org/doc/stable/ • Matplotlib: matplotlib.org/stable/ • Pandas: pandas.pydata.org/docs/

Material didático autoral EP. A utilização de código e materiais de terceiros está sujeita às respectivas licenças.